

◇ Git & ● GitHub

Resumen de Contenidos

IIC2233 — Programación Avanzada

Resumen elaborado por el equipo de ayudantía, en base al material oficial del curso

Contents

1	¿Qué es Git?	2
1.1	¿Qué es GitHub?	2
1.2	Repositorios del curso	2
2	Instalación y configuración	2
2.1	Verificar si Git ya está instalado	2
2.2	Instalación según sistema operativo	3
2.3	Configuración inicial obligatoria	3
3	Entendiendo Git: las 4 ubicaciones	3
4	Comandos básicos	4
4.1	git clone: descargar un repositorio	4
4.2	git pull: traer cambios del repositorio remoto	4
4.3	git add: preparar cambios	4
4.4	git commit: crear una versión	5
4.5	git push: subir cambios al remoto	5
4.6	Flujo de trabajo resumido	5
5	Entendiendo un commit	5
5.1	Analogía de la muñeca rusa (Matryoshka)	5
5.2	¿Por qué importa esto?	6
6	Autenticación	6
6.1	Token de acceso personal (<i>Personal Access Token</i>)	7
6.2	Primer uso y almacenamiento del token	7
7	Problemas comunes de Git	7
7.1	Repositorio desincronizado	7
7.2	Subir archivos muy pesados a GitHub	8
7.3	Solución rápida y segura ante cualquier problema	8
7.4	Buenas prácticas para prevenir problemas	8
8	Resumen de comandos	9
9	Ejercicios de práctica	9

1 ¿Qué es Git?

Git es un **sistema distribuido de control de versiones**, es decir, una herramienta que permite *guardar, gestionar y rastrear* el historial de cambios de un proyecto a lo largo del tiempo. Gracias a Git, es posible:

- Mantener copias de seguridad de un proyecto a medida que se realizan cambios.
- Volver a versiones anteriores del código si algo se rompe.
- Trabajar de forma colaborativa sin sobrescribir el trabajo de otras personas.

En este curso, Git se utiliza principalmente para **descargar contenidos y enunciados**, y para **entregar las evaluaciones** a lo largo del semestre.

1.1 ¿Qué es GitHub?

GitHub es una **plataforma en la nube** que utiliza Git como sistema de control de versiones subyacente. Se puede pensar como un “Google Drive” especializado en proyectos de software:

- Los archivos se almacenan en la nube y son accesibles desde cualquier lugar.
- A diferencia de Google Drive, está optimizado para gestión de código y trabajo colaborativo.
- Es, hoy en día, el estándar de la industria del software.

★ Diferencia clave

Git es la herramienta (el motor de control de versiones) que corre en tu computador. **GitHub** es un servicio web que aloja repositorios remotos usando Git por detrás. Se puede usar Git sin GitHub, pero en este curso los usaremos siempre juntos.

1.2 Repositorios del curso

En IIC2233 se trabaja con tres repositorios distintos en GitHub:

- **Syllabus**: contiene las clases, ayudantías y tareas del curso.
- **Contenidos**: material de estudio (el curso usa *Flipped Classroom*, por lo que hay que estudiarlos por cuenta propia antes de clases).
- **Repositorio personal**: donde cada estudiante sube sus tareas y actividades.

★ Discussions

Cada repositorio tiene una pestaña **Discussions**, que funciona como el foro del curso para preguntas de instalación, contenidos, tareas, etc. Antes de preguntar: busca en Google (idealmente en inglés) y revisa si alguien ya hizo la misma pregunta. **No se debe subir código de las tareas** al foro, y los ayudantes no responden entre las 20:00 del sábado y las 8:00 del lunes.

2 Instalación y configuración

2.1 Verificar si Git ya está instalado

Antes de instalar, se puede comprobar si Git ya está disponible en el sistema:

\$ Verificar instalación

```
git --version
```

Si aparece un número de versión (por ejemplo, `git version 2.9.2`), Git ya está instalado y no es necesario seguir los pasos de instalación.

2.2 Instalación según sistema operativo

- **Lubuntu / Linux (Debian-like)**, el entorno estándar del curso:

```
sudo apt update
sudo apt install git
```

- **Linux (Arch-like)**:

```
sudo pacman -S git
```

- **macOS**: se instalan primero las herramientas de desarrollo de Xcode y Homebrew, y luego Git:

```
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew
  /install/HEAD/install.sh)"
brew install git
```

- **Windows**: se descarga el instalador oficial y se siguen los pasos indicados por el asistente gráfico.

2.3 Configuración inicial obligatoria

La **primera vez** que se usa Git en un computador, es obligatorio registrar un nombre y un correo, que quedarán asociados a cada *commit* creado:

\$ Configuración de identidad

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@uc.cl"
```

! Importante

Si no se configura el nombre y el correo, el primer `git commit` fallará con un error del tipo fatal: `unable to auto-detect email address`. Esta configuración se hace **una sola vez** por computador.

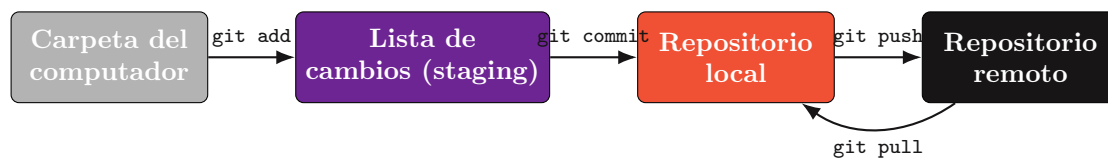
Opcionalmente, se puede habilitar el uso de colores en la terminal:

```
git config --global color.ui auto
```

3 Entendiendo Git: las 4 ubicaciones

Para usar Git correctamente basta con entender que un **repositorio** es un almacenamiento virtual de un conjunto de archivos que guarda *no solo el estado final*, sino **todas las versiones** que se han ido registrando en él.

Al trabajar con Git, los archivos de un proyecto pueden estar en **4 ubicaciones distintas**:



★ Idea central

Un proyecto “avanza” moviendo cambios de izquierda a derecha en el diagrama de arriba (`add` → `commit` → `push`), y se “actualiza” con los cambios de otros trayendo información de derecha a izquierda (`pull`).

4 Comandos básicos

Para este curso basta con dominar **5 comandos**: `clone`, `pull`, `add`, `commit` y `push`. Todos se ejecutan desde la terminal.

4.1 git clone: descargar un repositorio

Permite clonar (descargar) un repositorio remoto completo hacia el computador. Se ejecuta dentro de la carpeta donde se desea guardar el proyecto:

\$ `git clone`

```
git clone https://github.com/IIC2233/Syllabus.git
git clone https://github.com/IIC2233/contenidos.git
git clone https://github.com/IIC2233/<usuario>-iic2233-2026-2.git
```

Si el clonado fue exitoso, aparecerá una carpeta nueva por cada repositorio clonado.

4.2 git pull: traer cambios del repositorio remoto

Descarga hacia el repositorio local los cambios que existen en el repositorio remoto pero que aún no están en el computador.

\$ `git pull`

```
git pull
```

Se usa para descargar enunciados de tareas, contenidos nuevos, o sincronizar cambios cuando se trabaja desde más de un computador.

4.3 git add: preparar cambios

Le indica a Git la **intención** de incluir un archivo en la próxima versión (`commit`). El archivo pasa a la *lista de cambios (staging area)*, pero **todavía no queda guardado** en el repositorio.

\$ `git add`

```
git add README.md
```

Se puede llamar varias veces para ir agregando distintos archivos antes de crear el `commit`.

4.4 git commit: crear una versión

Toma todo lo que está en la lista de cambios y crea una **nueva versión** (commit) dentro del repositorio **local**, junto a un mensaje descriptivo y una marca de tiempo.

```
$ git commit
```

```
git commit -m "mensaje descriptivo explicando los cambios"
```

Tras este paso, el archivo ya forma parte del historial del repositorio local, pero aún **no ha sido enviado** al repositorio remoto.

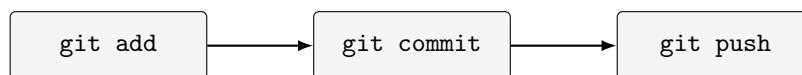
4.5 git push: subir cambios al remoto

Publica en el repositorio remoto (GitHub) todos los commits del repositorio local que todavía no se han subido.

```
$ git push
```

```
git push
```

4.6 Flujo de trabajo resumido



Comando	Qué hace
git clone	Descarga un repositorio remoto completo al computador.
git pull	Trae al repositorio local los cambios nuevos del remoto.
git add	Marca archivos para incluirlos en el próximo commit.
git commit	Crea una nueva versión con los cambios marcados.
git push	Sube los commits locales al repositorio remoto.

5 Entendiendo un commit

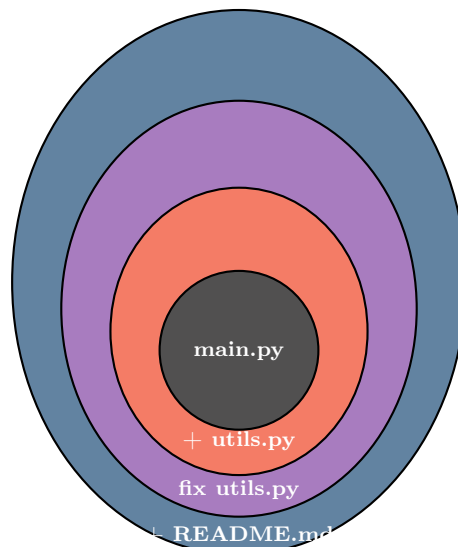
? Pregunta frecuente

“Si subo el README.md al final, ¿al descargar el último commit solo obtengo el README?”

Respuesta: ¡No! Cada commit guarda una **foto completa** (*snapshot*) del proyecto en ese momento: incluye **todos** los archivos que existían al hacer el commit, no solo los que se agregaron en ese commit puntual.

5.1 Analogía de la muñeca rusa (Matryoshka)

Cada commit puede pensarse como una **muñeca rusa**: los commits se guardan uno dentro de otro, en orden cronológico. La muñeca más grande es el último commit, y dentro de ella están contenidas (conceptualmente) todas las anteriores.



commit 4 (último): contiene todo lo anterior

1. **Commit 1** (muñeca más pequeña): solo contiene `main.py`.
2. **Commit 2**: incluye `main.py` y agrega `utils.py`.
3. **Commit 3**: no agrega archivos nuevos, corrige un error en `utils.py`.
4. **Commit 4** (el más grande): contiene todo lo anterior, más un nuevo archivo `README.md`.

Cada nueva “muñeca” encapsula todo lo que contenía la anterior, más los archivos agregados o modificados en ese momento.

! Ojo

Un commit es una **foto** del proyecto, no una acumulación infinita. Si se elimina un archivo y luego se hace commit, esa nueva foto **ya no lo incluye** (aunque el archivo sigue siendo recuperable desde commits anteriores del historial).

5.2 ¿Por qué importa esto?

Al hacer `git clone`, Git **no** entrega solo el último archivo modificado: entrega **todo el proyecto** hasta el punto de ese commit, incluyendo todos los archivos previos. Esto es la base del control de versiones: permite volver atrás, comparar versiones, o recuperar cualquier archivo de un commit anterior, porque cada commit es un estado **completo**, no un fragmento.

6 Autenticación

Al trabajar con repositorios de GitHub (especialmente los privados), es necesario **autenticarse** para poder leer o escribir en ellos. Existen distintos mecanismos, dependiendo del sistema operativo y la versión de Git instalada:

- **Usuario y token de acceso personal** directamente en la terminal (lo habitual en Ubuntu, ya que Git instalado con `apt` no incluye otros mecanismos).
- **Inicio de sesión por navegador web**, disponible en instalaciones con *Git Credential Manager* (por ejemplo, Git para Windows).
- **Llave SSH** (no se utiliza en este curso).

6.1 Token de acceso personal (*Personal Access Token*)

Es la alternativa al uso de contraseñas para autenticarse en GitHub desde la terminal. Se exige para administrar **repositorios privados**.

- **Creación:** se accede a `https://github.com/settings/tokens/new` y se elige un token de tipo *classic*.
- Se debe completar:
 1. Una **descripción** (*Note*) del token.
 2. Una **fecha de expiración** (idealmente posterior al cierre del semestre).
 3. El **scope** **repo** en *Select scopes*, que otorga acceso a repositorios privados (selecciona automáticamente sus subcasillas).
- Al presionar **Generate token** aparecerá un texto que comienza con "ghp_": ese es el token.

! Muy importante

El token **solo se muestra una vez** al momento de generarlo. Si se pierde, hay que generar uno nuevo repitiendo todo el proceso. Debe guardarse en un lugar seguro.

6.2 Primer uso y almacenamiento del token

La primera vez que se usa (por ejemplo, al hacer `git clone` del repositorio personal), Git pedirá **usuario** y luego el **token** como contraseña. El token no se mostrará en pantalla al escribirlo/pegarlo (por seguridad), pero esto es normal.

Por defecto, Git pedirá el token **cada vez** que se haga `clone`, `pull` o `push`. Para evitar esto, se puede guardar con:

\$ Guardar credenciales

```
git config --global credential.helper store
```

! Advertencia de seguridad

Con esta configuración, el token queda guardado en **texto plano** en el archivo `~/.git-credentials` del usuario. **No usar esta opción en computadores compartidos.**

7 Problemas comunes de Git

Si se usa Git de forma ordenada (`add/commit/push` frecuentes desde un solo lugar), no debería haber mayores problemas. Sin embargo, existen dos errores muy frecuentes entre estudiantes.

7.1 Repositorio desincronizado

Ocurre cuando se edita el repositorio desde **dos orígenes distintos** sin sincronizar antes (por ejemplo, editar el `README.md` desde la web de GitHub y también desde el computador). Git no puede resolver el orden de los commits y **bloquea el push** hasta que se resuelva el conflicto.

\$ Solución: combinar historiales

```
git config --global pull.rebase false
git pull
```

```
git push
```

Si al hacer `git pull` se abre un editor de texto pidiendo un mensaje de commit de combinación, basta con guardar y salir (en `nano`: `Ctrl+O`, `Enter`, `Ctrl+X`).

Si ambas versiones editaron **archivos distintos**, esto es suficiente para sincronizar todo. Pero si editaron **el mismo archivo**, aparece un error más delicado, el **merge conflict**:

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

! No recomendado

Resolver un *merge conflict* manualmente es un proceso **delicado** que puede **arruinar archivos de forma permanente** si se hace mal. El curso **no recomienda** intentarlo desde la terminal siguiendo tutoriales de internet.

7.2 Subir archivos muy pesados a GitHub

GitHub **prohíbe automáticamente** subir archivos individuales que pesen más de **100 MB**. El problema es que `git add` y `git commit` sí lo permiten, pero al hacer `git push` este falla.

! Ojo con el historial

Aunque luego se elimine el archivo pesado y se vuelva a hacer `add`, `commit` y `push`, el **push seguirá fallando**: como Git guarda todo el historial, el hecho de que el archivo alguna vez existió sigue registrado y bloquea la subida.

Archivos de código no suelen pesar mucho; para fotos, videos u otros archivos pesados se debe usar otro medio de almacenamiento, no GitHub.

7.3 Solución rápida y segura ante cualquier problema

★ Receta general anti-pánico

1. En **otra ubicación** del computador, clonar de nuevo el repositorio personal con `git clone` (esto trae una copia limpia de lo que hay en GitHub).
2. Desde la copia **original** (la que tiene problemas), copiar y pegar en la nueva copia todos los archivos que se quieran conservar.
3. En la **nueva copia**, hacer `git add`, `git commit` y `git push`.
4. Verificar en GitHub que todo se haya subido correctamente, y recién ahí **eliminar la copia antigua** con problemas.

7.4 Buenas prácticas para prevenir problemas

- Al terminar de trabajar, hacer siempre `git add`, `git commit` y `git push`.
- **No editar** archivos directamente desde la página web de GitHub.
- Si se sabe que el repositorio remoto cambió, hacer `git pull` lo antes posible.
- Ante cualquier error desconocido, preferir la “receta general” de la sección anterior antes que tutoriales externos con comandos no vistos en el curso.

- Hacer avances **frecuentes** para evitar entregas fuera de plazo por errores de Git.
- Ante cualquier duda, preguntar a tiempo en el foro (*Discussions*) del curso.

8 Resumen de comandos

Comando	Descripción
<code>git --version</code>	Verifica si Git está instalado y su versión.
<code>git config --global user.name "..."</code>	Configura el nombre asociado a los commits.
<code>git config --global user.email "..."</code>	Configura el correo asociado a los commits.
<code>git config --global credential.helper store</code>	Guarda el token para no pedirlo cada vez.
<code>git config --global pull.rebase false</code>	Define la estrategia <i>merge</i> para reconciliar ramas divergentes.
<code>git clone <url></code>	Descarga un repositorio remoto completo.
<code>git pull</code>	Trae cambios nuevos desde el repositorio remoto.
<code>git add <archivo></code>	Agrega un archivo a la lista de cambios (staging).
<code>git commit -m "mensaje"</code>	Crea una nueva versión con los cambios en staging.
<code>git push</code>	Sube los commits locales al repositorio remoto.

◇ ¡Éxito! •

Con estos 5 comandos y buenas prácticas de sincronización, tendrás todo lo necesario para trabajar con Git y GitHub durante el curso.

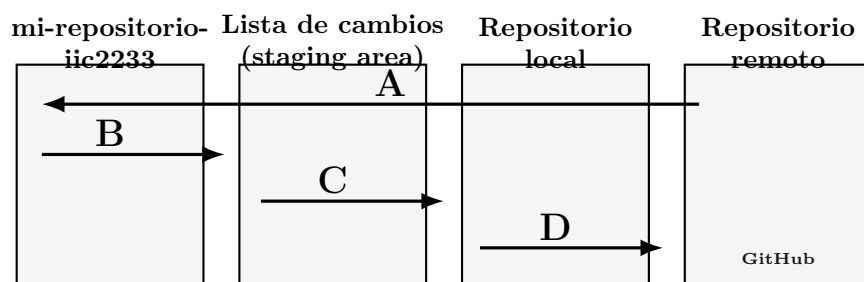
9 Ejercicios de práctica

Ejercicio 1

Dificultad: 🐱🐱🐱🐱🐱

(3 de 5 gatitos negros)

A partir del siguiente diagrama de flujo de trabajo en **Git**, y considerando lo visto en clases, cada flecha (A, B, C y D) representa una operación entre los distintos espacios de trabajo.



¿Cuál de las siguientes alternativas asigna **correctamente** cada letra con su operación correspondiente? Suponga que el repositorio ya existe en **GitHub**, que no hay problemas de conexión y que todas las operaciones se ejecutan correctamente.

A)

A: `git pull`
B: `git push`
C: `git add README.md`
D: `git commit -m "Mucho éxito"`

B)

A: `git pull`
B: `git commit -m "Mucho éxito"`
C: `git add README.md`
D: `git push`

C)

A: `git push`
B: `git add README.md`
C: `git commit -m "Mucho éxito"`
D: `git pull`

D)

A: `git pull`
B: `git add README.md`
C: `git commit -m "Mucho éxito"`
D: `git push`

E)

A: `git pull`
B: `git add README.md`
C: `git push`
D: `git commit -m "Mucho éxito"`

✓ Respuesta y desarrollo paso a paso

Alternativa correcta: D)

Paso 1 — Identificar la dirección de cada flecha. En el diagrama, las flechas B, C y D apuntan hacia la **derecha** (desde *mi-repositorio-iic2233* hacia el *repositorio remoto*), mientras que la flecha A es la única que apunta hacia la **izquierda** (desde el *repositorio remoto* de vuelta hacia *mi-repositorio-iic2233*).

Paso 2 — Flecha A: repositorio remoto → mi-repositorio. La única operación de las 5 vistas en el curso que trae información **desde** el repositorio remoto **hacia** el computador es `git pull`. Por lo tanto, **A: `git pull`**.

Paso 3 — Flecha B: mi-repositorio → lista de cambios (staging). La operación que mueve un archivo desde la carpeta del computador hacia la lista de cambios es `git add`. Por lo tanto, **B: `git add README.md`**.

Paso 4 — Flecha C: staging → repositorio local. La operación que toma lo que está en la lista de cambios y crea una nueva versión dentro del repositorio local es `git commit`. Por lo tanto, **C: `git commit -m "Mucho éxito"`**.

Paso 5 — Flecha D: repositorio local → repositorio remoto. La operación que publica los commits del repositorio local en el repositorio remoto (GitHub) es `git push`. Por lo tanto, **D: `git push`**.

Conclusión. La secuencia correcta es A: pull, B: add, C: commit, D: push, que corresponde exactamente a la **alternativa D)**.

- La alternativa **A** invierte B y D (pone `push` donde va `add`), lo cual no calza con la dirección de las flechas.
- La alternativa **B** invierte B y C (`commit` antes que `add`), pero `commit` no puede ir antes que `add` en el flujo mostrado.
- La alternativa **C** invierte A y D, asignando `push` a la flecha que apunta hacia la izquierda, lo cual es incorrecto.
- La alternativa **E** invierte C y D (`push` antes que `commit`), lo que no corresponde al orden de las cajas en el diagrama.

Dificultad: 

(1 de 5 gatitos negros)

¿Por qué se **recomienda** utilizar un sistema de control de versiones como **Git** durante el desarrollo de código?

- A) Permite mantener un respaldo del trabajo realizado.
- B) Permite recuperar versiones anteriores del proyecto.
- C) Facilita la colaboración entre múltiples desarrolladores.
- D) Permite llevar un registro ordenado de los cambios realizados.
- E) Todas las anteriores.

✓ Respuesta y desarrollo paso a paso

Alternativa correcta: E) Todas las anteriores

Paso 1 — Revisar la alternativa A. Como se vio en la sección 1, Git permite **mantener copias de seguridad** de un proyecto a medida que se hacen cambios: si algo se pierde o se rompe en el computador, el trabajo sigue disponible en el historial de commits (y en el repositorio remoto). Esto hace que A sea **verdadera**.

Paso 2 — Revisar la alternativa B. Cada **commit** guarda una foto completa del proyecto en un momento dado (ver la analogía de la muñeca rusa). Esto permite **volver a versiones anteriores** del código en cualquier momento. Por lo tanto, B es **verdadera**.

Paso 3 — Revisar la alternativa C. Git es un sistema **distribuido**, pensado para que varias personas trabajen sobre el mismo proyecto sin sobrescribir el trabajo ajeno, sincronizando cambios mediante **pull** y **push**. Esto **facilita la colaboración**, por lo que C es **verdadera**.

Paso 4 — Revisar la alternativa D. Cada commit queda asociado a un mensaje, una fecha y un autor, lo que constituye un **registro ordenado y cronológico** de todos los cambios realizados en el proyecto. Por lo tanto, D también es **verdadera**.

Paso 5 — Concluir. Como las alternativas A, B, C y D son todas afirmaciones correctas y no son mutuamente excluyentes (todas describen beneficios reales de usar Git), la alternativa que engloba correctamente la pregunta es **E) Todas las anteriores**.

Ejercicio 3

Dificultad: 

(3 de 5 gatitos negros)

Un programador está usando **Git** para manejar las versiones de su código. La siguiente figura muestra en la izquierda las carpetas y archivos presentes en su código, marcando con un ***** a los elementos que fueron modificados desde el *commit* anterior. En la derecha, se muestra la sucesión de comandos de Git utilizados para subir cambios al repositorio remoto.

```
1 a.py*
2 data/
3   b.txt*
4 outputs/
5   c.txt*
6   d.txt*
7 e.txt
8 f.txt*
9 g.uwu*
```

```
1 git add a.py
2 git add b.txt
3 git add /outputs/
4 git add e.txt
5 git add g.uwu
6 git commit -m "Mi entrega"
7 git add f.txt
8 git push
```

En base a lo anterior, ¿cuál de las siguientes aseveraciones es **verdadera**?

- A) Los cambios no se subirán al repositorio remoto, ya que faltó el comando `git status` al inicio de la secuencia.
- B) El archivo `b.txt` será subido en el nuevo commit ya que Git busca de forma recursiva dentro de carpetas hasta encontrar el archivo.
- C) Los archivos `c.txt` y `d.txt` serán subidos ya que se usó un comando para agregar la carpeta completa que los contenía.
- D) El archivo `g.uwu` no será subido ya que tiene una extensión inválida que no es usada por ningún programa.
- E) Los archivos que contendrá el commit recién subido serán solamente `a.py` y `g.uwu`.

✓ Respuesta y desarrollo paso a paso

Alternativa correcta: E)

Paso 1 — Descartar la alternativa A. El comando `git status` es puramente **informativo**: muestra el estado del repositorio (qué está modificado, en staging, etc.), pero **no es necesario** para que `add`, `commit` o `push` funcionen. Su ausencia no impide que los cambios se suban. **A es falsa.**

Paso 2 — Descartar la alternativa B. El archivo modificado está en `data/b.txt`, pero el comando ejecutado fue `git add b.txt`, **sin** el prefijo `data/`. Git **no busca de forma recursiva** un archivo por su nombre dentro de subcarpetas: solo interpreta rutas relativas a la carpeta actual. Como no existe un `b.txt` en la raíz del proyecto, este comando falla (`pathspec 'b.txt' did not match any files`) y `b.txt` **no queda en staging**. **B es falsa** (y además su justificación —que Git busca recursivamente— es conceptualmente incorrecta).

Paso 3 — Descartar la alternativa C. El comando usado fue `git add /outputs/`, con un **“/” inicial**. Ese slash al comienzo convierte la ruta en una ruta **absoluta** (partiendo desde la raíz del sistema de archivos), y no en la ruta **relativa** `outputs/` dentro del repositorio. Como esa carpeta absoluta normalmente no corresponde al repositorio, el comando falla y `c.txt` y `d.txt` **no quedan en staging**. **C es falsa.**

Paso 4 — Descartar la alternativa D. Git es **completamente agnóstico** a la extensión de los archivos: no valida ni restringe qué extensiones se pueden agregar. `git add g.uwu` funciona exactamente igual que con cualquier otro archivo, extensión “inválida” o no. **D es falsa.**

Paso 5 — Reconstruir qué queda efectivamente en staging antes del commit. Siguiendo la secuencia en orden:

- `git add a.py` → **éxito**, `a.py` queda en staging.

- `git add b.txt` → **falla** (Paso 2).
- `git add /outputs/` → **falla** (Paso 3).
- `git add e.txt` → éxito, pero `e.txt` **no tenía modificaciones** (no tiene *), por lo que no aporta ningún cambio nuevo.
- `git add g.uwu` → **éxito**, `g.uwu` queda en staging.

Paso 6 — El commit. Al ejecutar `git commit -m "Mi entrega"`, la nueva versión creada incorpora **solo las modificaciones que estaban en staging**: las de `a.py` y `g.uwu`. Como se vio en la Sección 5 (analogía de la muñeca rusa), el commit resultante sigue siendo una **foto completa** del proyecto —incluye también `e.txt` y las versiones *antiguas, sin modificar* de `b.txt`, `c.txt` y `d.txt`—, pero los únicos archivos cuyo **contenido nuevo** queda registrado en este commit son `a.py` y `g.uwu`.

Paso 7 — El add posterior al commit. `git add f.txt` se ejecuta **después** del commit, por lo que su modificación queda en staging pero **no forma parte de ningún commit** (no hay un segundo `git commit` tras esta línea).

Paso 8 — El push. `git push` sube al repositorio remoto **únicamente los commits ya creados**. En este caso, se sube el único commit existente —el que contiene las modificaciones de `a.py` y `g.uwu`—. El cambio en `f.txt` sigue en staging local y **no se sube**, al igual que las modificaciones no confirmadas de `b.txt`, `c.txt` y `d.txt`.

Conclusión. De los archivos modificados originalmente (marcados con *), los **únicos** cuyas modificaciones quedan efectivamente registradas en el commit que se sube al repositorio remoto son `a.py` y `g.uwu`. Esto hace que la **alternativa E sea verdadera**, y descarta a todas las demás.